

# CS 112

Introduction to Data Structures

---

WEEK 04

## PyList: Dynamic Arrays

**Eric Araújo**

Calvin University · Fall 2026

# This Week

---

- ➊ Why a plain array isn't enough
- ➋ `new` and `delete`
- ➌ The stack and the heap
- ➍ Building a dynamic array
- ➎ Growing when full
- ➏ Destructors: now you need one
- ➐ Copy constructors: shallow vs deep **KEY**

# Arrays: The Good and the Bad

---

## Good



- Simple to declare
- Indexing is fast: one multiply and add
- Contiguous, so the cache loves them

## Bad



- Size fixed at **compile time**
- Can't grow when you need one more
- No idea how many slots are actually used

This week we will build a Python-like list that grows if needed.

# Dynamic Memory Allocation

---

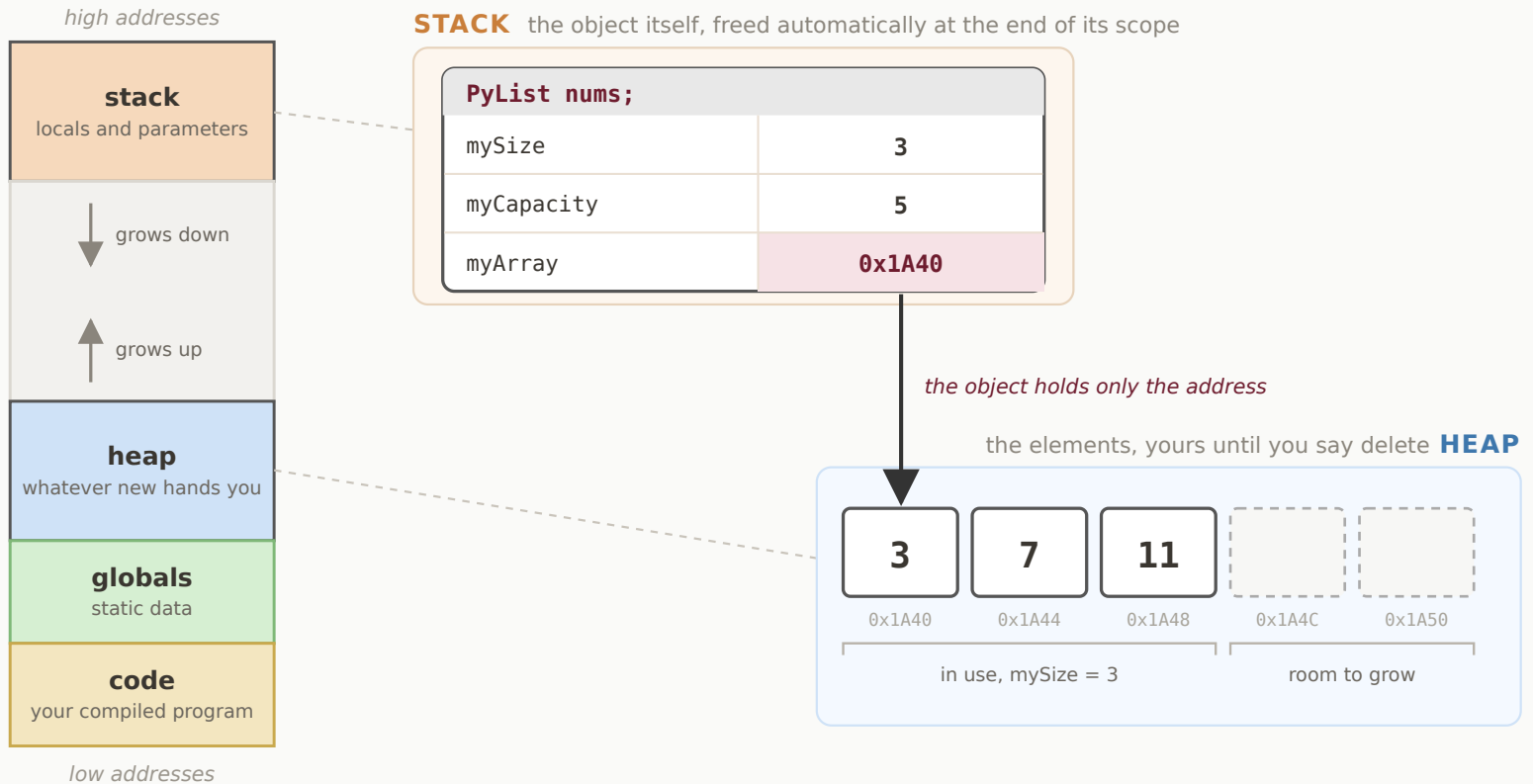
```
int *arr = new int[capacity];    // ask the OS for space
                                   // and a delete to match

arr[0] = 42;                      // use it like an array

delete [] arr;                   // free the space afterwards
arr = nullptr;                  // don't keep the address
```

- `new` returns the **address** of the block
- The size can be a variable: decided while the program runs
- `delete []` for arrays, plain `delete` for single objects

# Stack and Heap



The object is on the stack. Its data is on the heap. Only the heap part needs a delete .

# A Dynamic Array

```
class PyList {
public:
    PyList();                // starts empty, with no array
    PyList(const PyList &original); // copy: builds its own array
    ~PyList();               // frees the array
    unsigned getSize() const; // how many items are in it
    void append(const Item &it); // add one more at the end
    Item& operator[](unsigned i); // v[i], on either side of an =
private:
    Item *myArray;           // address of the block, or nullptr
    unsigned mySize;         // slots used so far
    unsigned myCapacity;     // slots the block has room for
};
```

**size** is how many items you have. **capacity** is how many fit before it must grow.

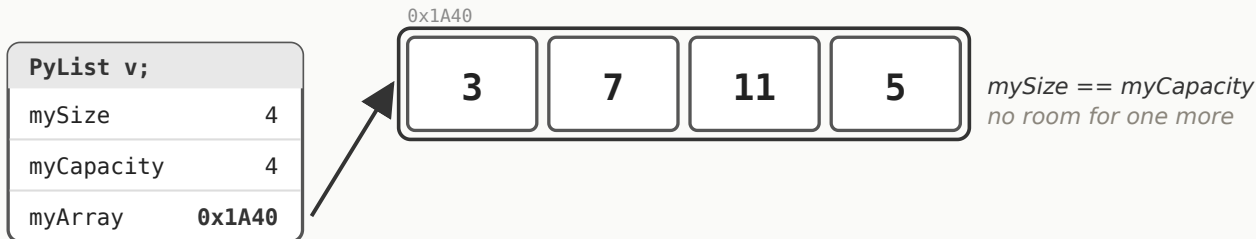
# Appending

```
void PyList::append(const Item &it) {
    if (mySize == myCapacity) {           // no room left
        myCapacity = (myCapacity == 0)
            ? 1                           // first append
            : myCapacity * 2;             // else double
        Item *bigger = new Item[myCapacity];
        for (unsigned i = 0; i < mySize; ++i) {
            bigger[i] = myArray[i];       // copy each one
        }
        delete [] myArray;                // free the old
        myArray = bigger;                 // keep the new
    }
    myArray[mySize] = it;                  // room now
    ++mySize;
}
```

# Growing, Step by Step

- 1 The array is full. There is no room for one more.

```
v.append(9);
```



# Growing, Step by Step

2 Ask the heap for a block twice the size.

```
v.append(9);
```

| PyList v;  |        |
|------------|--------|
| mySize     | 4      |
| myCapacity | 8      |
| myArray    | 0x1A40 |



*mySize == myCapacity  
no room for one more*

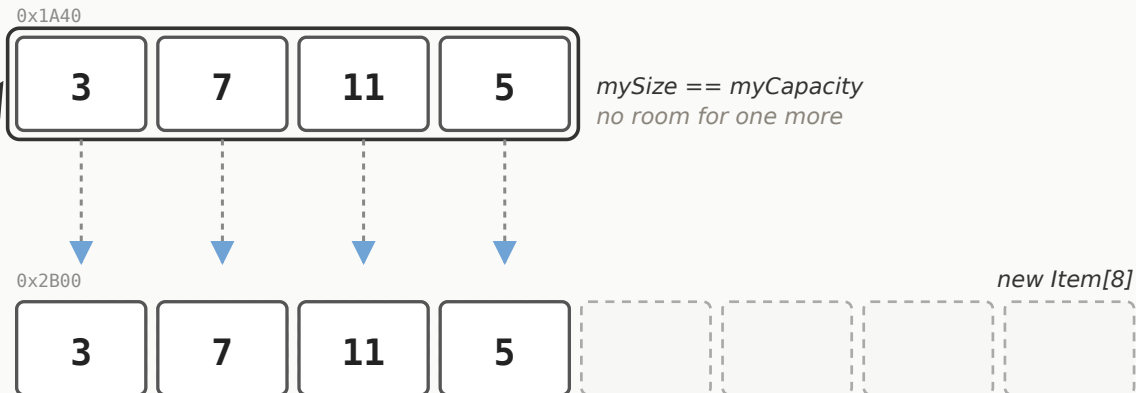


# Growing, Step by Step

## 3 Copy every item across.

`v.append(9);`

| PyList v;  |        |
|------------|--------|
| mySize     | 4      |
| myCapacity | 8      |
| myArray    | 0x1A40 |



# Growing, Step by Step

## 4 Free the old block.

```
v.append(9);
```

| PyList v;  |        |
|------------|--------|
| mySize     | 4      |
| myCapacity | 8      |
| myArray    | 0x1A40 |



*mySize == myCapacity  
no room for one more  
delete [] myArray*



# Growing, Step by Step

5 Point `myArray` at the new block.

```
v.append(9);
```

| PyList v;  |        |
|------------|--------|
| mySize     | 4      |
| myCapacity | 8      |
| myArray    | 0x2B00 |



*mySize == myCapacity  
no room for one more  
delete [] myArray*



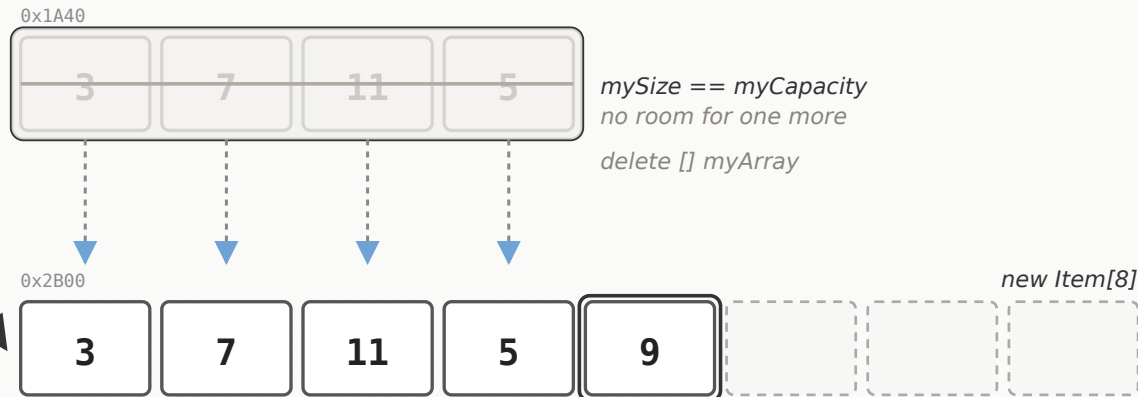
*myArray = bigger*

# Growing, Step by Step

6 Store the new item, and count it.

```
v.append(9);
```

| PyList v;  |        |
|------------|--------|
| mySize     | 5      |
| myCapacity | 8      |
| myArray    | 0x2B00 |



myArray = bigger

## TALK TO YOUR NEIGHBOR · TTYN

**Why double the capacity instead of adding one slot?**

- A.** Doubling uses less memory
- B.** Adding one would make every append copy the whole array
- C.** It doesn't matter, both are equally fast

# Destructor

---

```
PyList::~~PyList() {           // a ~ then the class name
    delete [] myArray;         // give the block back
    myArray = nullptr;         // and don't keep the address
}
```

- Destructors have no arguments or return type. You also never call them.
- They are called when the object loses its scope, and when you delete an object explicitly.
- Only one destructor per class, with no overload permitted.

# Rule of Thumb

---

1

`PyList`  
`v;`

puts an object on the stack. There is no block yet.

```
void f() {  
    PyList v;  
    v.append(3);  
}
```

THE STACK

| PyList v;  |         |
|------------|---------|
| mySize     | 0       |
| myCapacity | 0       |
| myArray    | nullptr |

THE HEAP

# Rule of Thumb

- 2 `v.append(3);` asks the heap for a block. `myArray` holds its address.

```
void f() {  
    PyList v;  
    v.append(3);  
}
```

THE STACK

| PyList v;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

THE HEAP



# Rule of Thumb

3 The closing brace destroys `v`, so a destructor runs.

```
void f() {  
    PyList v;  
    v.append(3);  
} // v is destroyed here
```

THE STACK

| PyList v;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

*~PyList() runs  
the one C++ wrote for you*

THE HEAP

0x1A40

3

*new Item[1]*

# Rule of Thumb

4

The one C++ wrote destroys the three members, the pointer included.

```
void f() {  
    PyList v;  
    v.append(3);  
} // v is destroyed here
```

THE STACK



*~PyList() runs  
the one C++ wrote for you  
all three members, the pointer included*

THE HEAP



# Rule of Thumb

5 The block is still allocated, and nothing points at it.

```
void f() {  
    PyList v;  
    v.append(3);  
} // v is destroyed here
```

THE STACK



*~PyList() runs  
the one C++ wrote for you  
all three members, the pointer included*

THE HEAP



*new Item[1]*

**LEAK**

*still allocated  
and nothing points at it*

# Rule of Thumb

- 6 A destructor you write is what gives the block back.

```
void f() {  
    PyList v;  
    v.append(3);  
} // v is destroyed here
```

THE ONE YOU WRITE

```
PyList::~~PyList() {  
    delete [] myArray;  
}
```

THE STACK



`~PyList()` runs  
the one C++ wrote for you  
all three members, the pointer included

THE HEAP



given back

If your class calls `new`, you must define a **destructor**. C++ creates a destructor for you if you are not using `new`.

## TALK TO YOUR NEIGHBOR · TTYN

In which lines does a `PyList` destructor run?

```
void f() {                               // 1
    PyList *p = new PyList;             // 2
    PyList v;                           // 3
    v.append(3);                         // 4
    delete p;                           // 5
}
```

- A. Lines 2 and 3, where the two objects are built
- B. Lines 5 and 6
- C. Line 6 only, at the closing brace
- D. Line 5 only, because that is the only `delete`

## TALK TO YOUR NEIGHBOR · TTYN

After line 4, what does `b.myArray` hold?

```
PyList a;           // 1  
a.append(3);        // 2  
a.append(7);        // 3  
PyList b = a;       // 4
```

- A. The same address as `a.myArray`
- B. The address of a new block
- C. `nullptr`
- D. The two items themselves

# The Shallow Copy

1 `a` has one item, in a block of its own.

```
void f() {  
    PyList a;  
    a.append(3);  
    PyList b = a;  
    b[0] = 9;  
}
```

THE STACK

| PyList a;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

THE HEAP

0x1A40



# The Shallow Copy

2 b copies the three members. One block, two arrows.

```
void f() {  
    PyList a;  
    a.append(3);  
    PyList b = a;  
    b[0] = 9;  
}
```

THE STACK

| PyList a;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

| PyList b;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

THE HEAP

0x1A40



*the same address, not a second block*

# The Shallow Copy

3 A write through `b` changes what `a` reads.

```
void f() {  
    PyList a;  
    a.append(3);  
    PyList b = a;  
    b[0] = 9;  
}
```

THE STACK

| PyList a;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

| PyList b;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

THE HEAP

0x1A40



*a[0] reads 9 too*

*the same address, not a second block*

# The Shallow Copy

4 `b` is destroyed, and frees the block.

```
void f() {  
    PyList a;  
    a.append(3);  
    PyList b = a;  
    b[0] = 9;  
} // a and b are both destroyed here
```

THE STACK

| PyList a;  |        |
|------------|--------|
| mySize     | 1      |
| myCapacity | 1      |
| myArray    | 0x1A40 |

| PyList b;             |                   |
|-----------------------|-------------------|
| <del>mySize</del>     | <del>1</del>      |
| <del>myCapacity</del> | <del>1</del>      |
| <del>myArray</del>    | <del>0x1A40</del> |

THE HEAP



*a[0] reads 9 too*

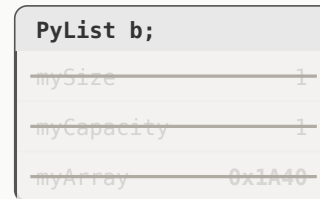
*the same address, not a second block*

# The Shallow Copy

5 `a` is destroyed, and frees it again.

```
void f() {  
    PyList a;  
    a.append(3);  
    PyList b = a;  
    b[0] = 9;  
} // a and b are both destroyed here
```

THE STACK



THE HEAP

0x1A40



`a[0]` reads 9 too

freed by b

a frees it again

**CRASH**

the same address, not a second block

`free(): double free detected`

## TALK TO YOUR NEIGHBOR · TTYN

### What happens when this runs?

```
PyList a;           // 1
a.append(3);         // 2
PyList b = a;        // 3
b[0] = 9;            // 4
cout << a[0] << endl; // 5
// 6  a and b both go out of scope
```

- A. Prints 3, then each block is freed once
- B. Prints 9, then one block is freed twice
- C. Prints 3, and one block leaks
- D. Prints 9, then line 3 fails to compile

# The Copy Constructor

---

A constructor whose parameter is **another object of the same class**.

```
PyList::PyList(const PyList &orig) {    // orig is the one we copy FROM
    mySize      = orig.mySize;          // same count
    myCapacity  = orig.myCapacity;      // same room
    myArray     = new Item[myCapacity];  // our OWN block, a new address
    for (unsigned i = 0; i < mySize; ++i) {
        myArray[i] = orig.myArray[i];    // copy the values across
    }
}
```

# Three Times It Copies

```
PyList b = a;                                // 1. explicit copy

void show(PyList v);                          // 2. pass-by-value
show(a);

PyList makeList() {                           // 3. return by value
    PyList local;
    return local;
}
```

Case 2 is why week 1 said *pass big objects by const reference*. Every by-value call to `show()` copies the whole heap block.

# Returning a Reference: `operator[]`

```
Item& PyList::operator[](unsigned i) {  
    return myArray[i];           // a reference, not a copy  
}
```

```
v[2] = 99;           // works because v[2] IS the slot
```

Return `Item` and the call hands back a copy, and `v[2] = 99` does not compile at all. Returning `Item&` makes the call another name for the element.

# The `this` Pointer

---

## Python

```
def set_first(self, v):  
    self.my_first = v
```

## C++

```
void setFirst(Item v) {  
    this->myFirst = v;  
}
```

C++ passes the object silently as `this` : a pointer to the object the method was called on. You rarely write it, but it's there, and `this->x` is just `(*this).x`.

# Week 04 Recap

---

- `new` gets heap memory at runtime; every `new` needs one `delete`
- **size** vs **capacity** is what makes append cheap most of the time
- Growing means allocate, copy, delete, repoint: linear, so we double
- If your class calls `new`, it needs a **destructor**
- The default copy is **shallow**: aliasing plus a double delete
- Write a copy constructor that allocates its own block

Next: a05 →

